

Documentação do Código de Algoritmos de Ordenação

1. Início

Neste documento, descreve-se a implementação dos algoritmos de ordenação QuickSort, MergeSort e BubbleSort em Java, incluindo a geração de listas de teste para diferentes cenários, a execução de testes de desempenho comparativo e a realização de gráficos com a evolução dos tempos de execução.

2. Classe SortingAlgorithms

A classe SortingAlgorithms contém três implementações de algoritmos de ordenação: QuickSort, MergeSort e BubbleSort. Cada método é responsável por ordenar um array de inteiros utilizando a abordagem correspondente.

2.1. Método quickSort

Ordena um array de inteiros utilizando o algoritmo QuickSort. O método segue a abordagem de divisão e conquista, escolhendo um pivô e particionando a lista em dois subarrays.

2.2. Método mergeSort

Ordena um array de inteiros utilizando o algoritmo MergeSort. O método divide a lista em sublistas menores até alcançar listas unitárias e em seguida as mescla em ordem crescente.

2.3. Método bubbleSort

Ordena um array de inteiros utilizando o algoritmo BubbleSort. Percorre repetidamente a lista, trocando elementos adjacentes se estiverem fora de ordem, até que a lista esteja ordenada.

3. Classe TestCases

A classe TestCases é responsável por gerar listas de números inteiros para os testes de desempenho dos algoritmos de ordenação.

3.1. Método generateSmallList

Gera uma lista pequena contendo entre 10 e 100 elementos aleatórios.

3.2. Método generateMediumList

Gera uma lista média contendo entre 1.000 e 10.000 elementos aleatórios.

3.3. Método generateLargeList

Gera uma lista grande contendo 100.000 elementos ou mais, escolhidos aleatoriamente.

3.4. Método generateSortedList

Gera uma lista ordenada de inteiros de 1 até um tamanho específico, utilizada para avaliar o melhor caso dos algoritmos.

3.5. Método generateReverseSortedList

Gera uma lista de inteiros ordenada de forma reversa, representando o pior caso para alguns algoritmos.

3.6. Método generateDuplicatedList

Gera uma lista de inteiros contendo muitos elementos duplicados, testando a eficiência dos algoritmos com valores repetidos.

4. Classe SortingTest

A classe SortingTest é responsável por executar testes comparativos entre os algoritmos de ordenação, medindo o tempo de execução em diferentes cenários.

4.1. Método runSortingTests

Executa testes para cada algoritmo de ordenação utilizando uma lista específica, permitindo a comparação do desempenho.

4.2. Método testAlgorithm

Executa um algoritmo de ordenação, mede o tempo de execução e imprime os resultados no console.

5. Conclusão

O código fornece uma análise detalhada do desempenho dos algoritmos de ordenação, permitindo a comparação de suas eficiências em diferentes cenários. Com a medição do tempo de execução, é possível identificar qual algoritmo se comporta melhor dependendo do tipo de entrada.

6. Devolutiva no terminal pós execução

--- Teste com Lista Pequena ---

QuickSort: 0.239626 ms

MergeSort: 0.015228 ms

BubbleSort: 0.006713 ms

--- Teste com Lista Média ---

QuickSort: 0.432214 ms

MergeSort: 0.513435 ms

BubbleSort: 4.554122 ms

--- Teste com Lista Grande ---

QuickSort: 7.953426 ms

MergeSort: 16.130727 ms

BubbleSort: 9445.621890 ms

--- Teste com Lista Ordenada ---

QuickSort: 0.387811 ms

MergeSort: 0.053018 ms

BubbleSort: 0.001413 ms

--- Teste com Lista Reversamente Ordenada ---

QuickSort: 0.295229 ms

MergeSort: 0.049633 ms

BubbleSort: 0.242121 ms

--- Teste com Lista com Muitos Duplicados ---

QuickSort: 0.043972 ms

MergeSort: 0.065762 ms

BubbleSort: 0.491885 ms